

CSI243 Functional Programming

Lab 3 Recursion and Lists Continuation

Objectives

1. **Recursion**
2. **Applying recursion to basic lists**

Background / Scenario

Instead of using for or while loops, **recursion** is used for looping in Haskell. A recursive function **defines the function in terms of itself**. The function invokes itself only when a condition is met, as with an if/else/then expression. As in any form of loops, there has to be at least one case that terminates the recursion, and a recursive case that will cause the function to call itself.

So, to represent a recursive function, there should be two parts:

- The **base case** – defines the function for the first input values, terminates the recursion.
- The **recursive step** – defines the function for other input values in terms of itself, creating a loop

Recursion can be implemented using if then statements, or using a finite number of guards, where the first guard evaluates to true.

This lab continues from last week's lab. So if you didn't complete the lab, please do so!!!

For each of the questions below, there should be four parts:

- i. Specify the **signature** of the function, that is, the arguments and the return type of the function. Ensure appropriate function signatures, proper argument types, denoted properly in the specification
- ii. Give the **base** and the **recursive** definition of the function using guards where needed
- iii. Using an example, dry run the function
- iv. Then implement and test on Haskell

1. Define a function `atPosition` that takes a list and a number `y` and returns the element at position `y`, together with `y` as a pair (2-tuple). Remember that lists are indexed from 0. This means that calling `atPosition [3,5,2,10] 3` will return `(3,10)`. If `y` is less than 0 or `y` is greater than the length of the list, an appropriate error message should be displayed. To make life easy for you, there is an inbuilt list function `list!!y`, that returns the element at position `y` in list.
2. Given a list of integers and the integer `y`, the function `largerY` returns a list of all elements in the list which are larger than `y`. Assume the items can be arranged in some order.
As an example, `largerY [3,1,7,10] 4`, `y` is equals to 4, therefore it should return `[7,10]` as the answer. Calling `largerY` on an empty list should return an empty list.
3. Given a list of integers, the `positiveNumbers` function should takes the list and returns all the positive integers in the list as a new list.
Using `positiveNumbers [-1,5,-3,8]` should return `[5,8]`. Calling the function on an empty list should return an empty list.
4. Given a list of integers, the function `isSorted` should return `True` is the list is sorted in descending order, `False` otherwise. Calling `isSorted` on an empty list should always return `True`. Also, a list with only one element is sorted.
For example : `isSorted [5,3,1]` should return `True`, on the other hand, `isSorted [3,1,5]` should return `False`.